Правила разработки с ИИ-ассистентами

Параметр	Значение
Документ	Правила разработки с ИИ (документ 6 комплекта)
Версия	1.0
Дата	27.10.2026
Статус	На согласовании
Связанные документы	3 — Бэклог; 10 — Промпты; `AGENTS.md`; `prompts/`

1. Назначение

Единые правила работы с ИИ-ассистентами (ZCode, Claude Code и подобные) над этим репозиторием. Цель — предсказуемый результат: каждая история бэклога закрывается ассистентом с проверяемыми критериями, человек отвечает за решения.

2. Принципы

1. **История = задача.** Ассистенту передаётся одна история бэклога целиком (ID, критерии, ссылки на документы и контракт). Без истории — только исследования и вопросы.
2. **Тесты до кода.** Ассистент сначала пишет падающие тесты по критериям приёмки, потом реализацию.
3. **Контракт неприкосновенен.** `contracts/` и `docs/specs/` меняет только человек через PR (CODEOWNERS). Ассистент, которому «не хватает» поля в API, останавливается и задаёт вопрос.
4. **Человек решает, ассистент исполняет.** Архитектурные решения, безопасность, миграции данных — обязательное ревью технического лидера.
5. **Ничего лишнего.** Ассистент не добавляет зависимости, фичи и рефакторинг за пределами истории.

3. Содержимое AGENTS.md

Рабочие правила ассистента в репозитории — `AGENTS.md` (структура, команды, жёсткие правила, порядок выполнения задачи). Этот файл — операционная истина; документ 6 объясняет, почему правила такие.

4. Соглашения по коду

1. Сервер: CommonJS, Node 20+; домены в `src/domain/<имя>/` без HTTP-зависимостей; роуты в `src/api/` тонкие.
2. Время — только через `src/infra/clock.js`; `new Date()` в доменах запрещён (тесты на даты).
3. Случайность — только через `src/infra/random.js`.
4. Транзакция изменения + аудит — всегда вместе (`db.transaction`).

5. Фронтенд: ES-модули `public/src/`, клиент API `public/src/api/client.js` единственный, кто знает про fetch; вывод строк — через `esc()`.
6. Именование: таблицы во множественном числе, файлы доменов `kebab-case`.
7. Тесты: `test/unit/` (домены, без HTTP) и `test/api/` (HTTP). Имена файлов `*.test.js`.

5. Шаблон задачи для ассистента

```
## Задача {ID из бэклога}. {название}
Ссылки: docs/specs/03-backlog.md (эпик {E}), docs/specs/02-api-spec.md (раздел …),
contracts/openapi-vacation-api-v1.yaml (operationId …).
Контекст: {что уже есть; файлы; где смотреть}
Критерии приёмки:
- {критерий из бэклога}
Ограничения: {что нельзя трогать; версии; миграции}
Что на выходе: код + тесты + запись в CHANGELOG. Ревью ТЛ: {да/нет}.
```

6. Ветки, коммиты, слияние

- Ветка истории: `story/e03-02-request-lifecycle` от свежего `main`.
- Коммиты: `E03-02: переходы состояний заявки + тесты таблицей`.
- Одна история — один PR; в PR — чек-лист шаблона (что сделано, как проверено, решения).
- Слияние — squash; CI зелёный обязателен.

7. Готовность задачи

Ассистент сообщает о завершении только после: `npm run lint` и `npm test` зелёные; критерии приёмки отмечены со ссылками на тесты; CHANGELOG обновлён; «что осталось» перечислено честно.

8. Обязательное ревью человека

`src/auth/`, миграции, `scripts/migrate-stage0.mjs`, `contracts/`, `docs/specs/`, `.github/workflows/`, любые изменения правил доступа. Признак того, что ассистенту пора остановиться: нужно выбрать между вариантами поведения, не описанными в документах.

9. Что запрещено ассистенту

1. Менять контракт и документы комплекта.
2. Добавлять npm-зависимости.
3. Трогать `data.db` (это рабочие данные; тесты — временная БД).
4. Оставлять секреты, токены, пароли в коде, тестах и журналах.
5. «Улучшать» то, что не входит в историю.
6. Помечать историю выполненной без прохождения тестов.

10. Работа с данными

Ассистенту можно отдавать `seed.json` (демо-данные) и синтетические фикстуры. Рабочая БД и реальные персональные данные ассистенту не передаются. Генерация тестовых данных — только синтетические ФИО.

11. Цикл истории (см. prompts/base/)

P01 план истории → P02 тесты (красные) → P03 реализация → P04 самопроверка (lint/test) → P05 фикс CI → P06 независимое ревью вторым ассистентом → P07 критическое ревью человеком (для историй с ревью ТЛ) → P08 документация → P09 закрытие истории → P10/P11 разбор дефектов и блокеров → P12 закрытие итерации.